淘宝百亿补贴数据分析助手 Agent 实战

原创 天猫技术团队 大淘宝技术 2026年9月3日 18:55 浙江 16人



bybt-data-analysis-assistant是一个数据分析智能体，它能将自然语言提问转化为完整的数据分析流程，实现"你问它答"的智能化数据服务。该系统解决了传统数据分析中取数慢、口径混乱、归因依赖人工和知识不沉淀等痛点，提供NL2SQL取数、波动归因、血缘溯源、Python分析等全方位能力。其核心技术采用多Agent协同架构，通过意图识别、调度路由和ReAct推理循环实现智能决策，摒弃了简单的Text-to-SQL模式，创新性地采用NL2MDL2SQL路径，确保SQL生成的准确性和口径统一。系统构建了六层知识体系，从表结构到会话记忆分层管理，结合WrenAI语义层和Hologres加速查询，实现了从找表、写SQL、执行到深度分析的全流程自动化，大幅提升了数据分析效率和质量，使业务人员能像对话一样获取数据洞察。



1.1 它是什么

bybt-data-analysis-assistant 是一个**数据分析智能体**——你用自然语言提问，它自主完成从找表、写 SQL、执行查询到深度分析的全流程：

```
"昨天 GMV 多少？"          → 找表 → 生成 SQL → 执行 → 返回结果
"GMV 为什么跌了？"           → 波动归因 → 逐层下钻 → 因果链报告
"这个指标怎么计算的？"        → 血缘溯源 → 上游字段 → 计算逻辑
"各行业 GMV 做帕累托"        → 取数 → Python 分析 → 图表
```

它不是简单的 Text-to-SQL，而是一个**能思考、会选工具、能自我纠错的 Agent**。

1.2 解决什么痛点

痛点	传统方式	Agent 方式
取数慢	提需求 → 等开发 → 排期 → 交付	自然语言提问，秒级出数
口径乱	每个人写不同 SQL，WHERE 条件不一致	语义层 + 规则库保证口径统一
归因靠人工	手动拆维度，逐个跑 SQL 对比	自动诊断树，逐层下钻到根因
知识不沉淀	SQL 用完即弃，下次从零开始	自动暂存 → 审核入库 → 复用

1.3 能力速览

能力域	具体能力	状态
NL2SQL 取数	自然语言 → SQL → 执行 → 结果	✅ 生产可用
波动归因	自动诊断树 + 逐层下钻 + 因果链	✅ 生产可用
血缘溯源	字段级血缘 + 计算逻辑 + 上游依赖	✅ 生产可用
Python 分析	环比/帕累托/排名/透视 沙箱执行	✅ 生产可用
SQL 开发	生成/优化/审查 SQL	✅ 生产可用
元数据探查	表结构/分区/wiki/搜表	✅ 生产可用
ETL 源代码	查看 DataWorks 节点 SQL	✅ 生产可用
调度运维	实例/日志/报警/基线	✅ 生产可用
会话记忆	跨 Session 上下文 + 用户偏好	✅ 生产可用
SQL 自动暂存	对话产出 SQL → 暂存 → 审核入库	✅ 生产可用
阈值校准	阈值评估 + 混淆矩阵 + 回测	✅ 可用
多意图处理	一个问题含多个意图 → 多阶段执行	✅ 可用



2.1 Web 平台

启动服务：

```
./boot.sh
```

访问地址：

地址	用途
`http://<IP>:4173`	前端对话界面
`http://<IP>:8000/docs`	后端 API 文档
`http://<IP>:8000/health`	健康检查

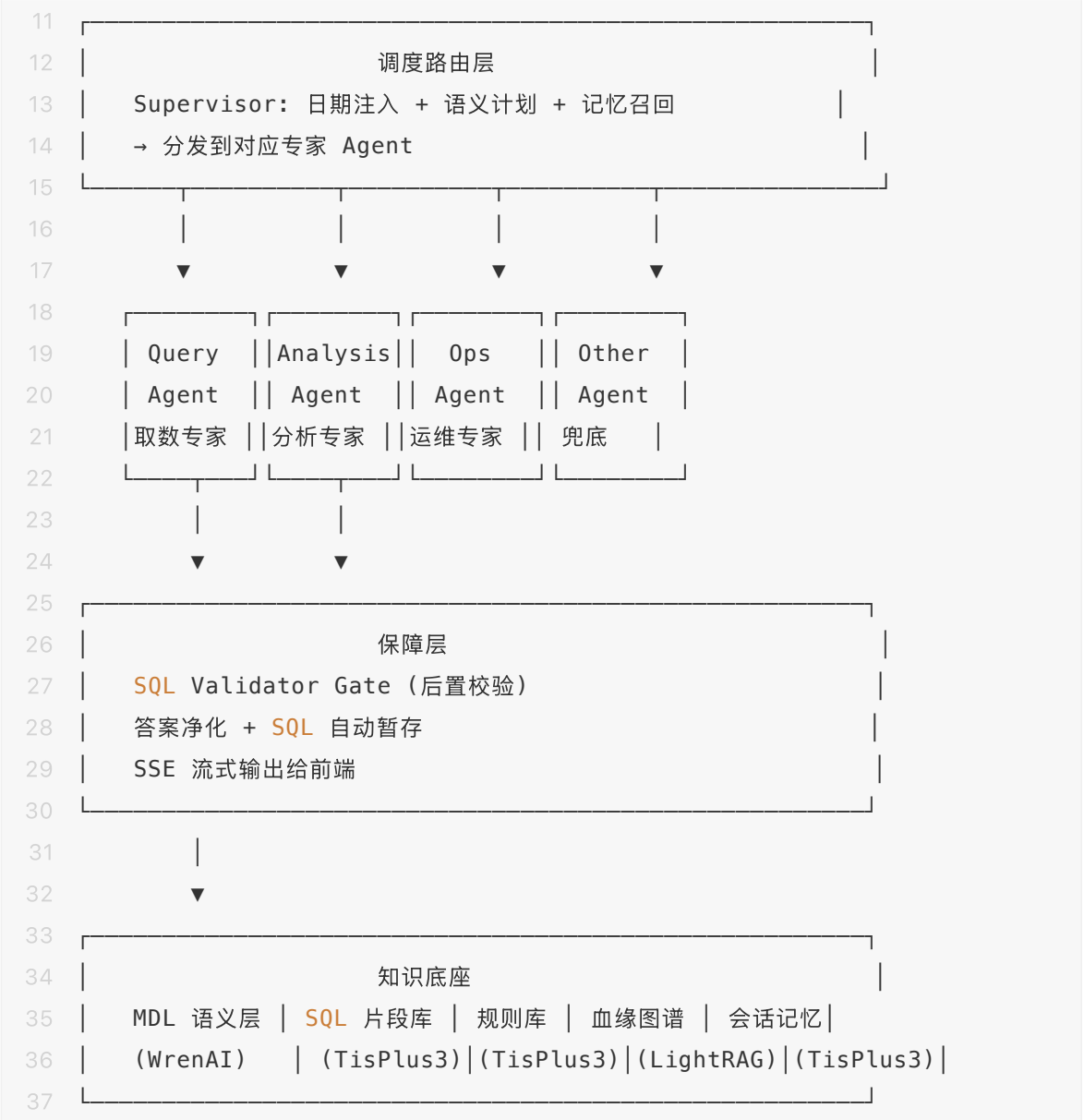
对话示例：

你输入	Agent 做什么
"昨天 GMV 多少"	找表 → 生成 SQL → 执行 → 返回数值
"GMV 为什么跌了 20%"	归因诊断树 → 逐层拆解 → 因果链报告
"这个字段怎么算的"	查血缘图谱 → 上游字段 → 计算表达式
"各行业 GMV 做帕累托分析"	取数 → Python 沙箱 → 帕累托图
"帮我写个查昨天 GMV 的 SQL"	生成 SQL → 校验 → 返回（不执行）
"这个表的 ETL 源代码"	查 DataWorks 节点 → 返回 SQL 代码



3.1 架构全景图

```
用户自然语言提问
    │
    ▼
┌──────────────────────────────────────────────────────┐
│                    意图识别层                        │
│   Intent Agent: LLM 单次调用 → JSON 意图参数包       │
│   (意图分类 + 时间解析 + 指标识别 + 维度推断)        │
└──────────────────────┬───────────────────────────────┘
                       │
                       ▼
```


调度路由层
Supervisor: 日期注入 + 语义计划 + 记忆召回
→ 分发到对应专家 Agent
Query Agent
取数专家
Analysis Agent
分析专家
Ops Agent
运维专家
Other Agent
兜底
保障层
SQL Validator Gate (后置校验)
答案净化 + SQL 自动暂存
SSE 流式输出给前端
知识底座
MDL 语义层
(WrenAI)
SQL 片段库
(TisPlus3)
规则库
(TisPlus3)
血缘图谱
(LightRAG)
会话记忆
(TisPlus3)

3.2 技术栈一览

层级	技术	选型理由
LLM	Qwen (DashScope)	阿里内部主力模型，中文理解强，函数调用稳定
轻量任务	qwen-turbo	意图识别、语义计划，速度快成本低
向量模型	text-embedding-v3	DashScope 向量模型，1024 维，中文语义检索效果好
Agent 框架	LangChain	`create_agent` 原生支持 ReAct 循环 + SSE 流式输出
多 Agent 编排	LangGraph	Supervisor 模式多 Agent 协同
语义层	WrenAI (wren-langchain)	MDL 建模 + NL2SQL + dry_plan
向量检索	TisPlus3 (Ha3)	阿里内部向量检索服务，支持 SQL 直连，延迟低
知识图谱	LightRAG	轻量级 RAG 知识图谱，支持 local/hybrid 检索
查询引擎	MaxCompute (ODPS)	阿里大数据计算平台
元数据	DataWorks API	表结构/分区/wiki/调度节点
后端	FastAPI + Uvicorn	Python 异步 Web 框架
前端	React + Ant Design + Vite	知识管理 + 对话平台
部署	boot.sh	一键启动（venv + 双进程 + 健康检查）

设计要点：不同环节用不同规格的模型——重活用 qwen-plus，轻活用 qwen-turbo，控制成本和延迟。

3.3 核心设计决策

决策	选择	一句话理由
SQL 生成方式	NL2MDL2SQL（而非直接 Text-to-SQL）	语义层校验，防止幻觉字段和口径错误
Agent 编排	多 Agent 协同（而非单 Agent）	每个专家 Agent 工具少、指令清晰，选择更准
推理模式	ReAct（而非固定 Pipeline）	自主决定调什么工具、传什么参数、何时收敛
知识存储	六层知识体系	从表结构到会话记忆，分层管理、分层复用

4.1 意图识别：入口层

意图识别是 Agent 的**大脑前哨**——在 Agent 执行前，先用一次 LLM 调用解析出标准化参数包：

```
{
"intent_type":"attribution",
"confidence":0.95,
"metric_field":"gmv_amt",
"metric_name":"GMV",
"time_label":"昨天 vs 前天",
"curr":"2026-06-14",
"prev":"2026-06-13",
"scope":"all"
}
```

一次调用同时完成：

维度	说明	示例
意图分类	是取数？归因？血缘？运维？	"为什么跌了" → attribution
时间解析	"昨天" → 具体日期	curr: 2026-06-14, prev: 2026-06-13
指标识别	用户说的 GMV 对应哪个字段	metric_field: gmv_amt
维度推断	是否需要按某维度拆解	scope: all / industry / channel

降级策略：LLM 调用失败时，自动回退到关键词匹配 + 规则解析。

多意图检测：一个问题含多个意图时，主意图放 `intent_type`，次要意图放 `secondary_intents` 数组，支持组合如 `table_meta + data_query`（查表结构 + 写查询）。

4.2 多 Agent 协同：调度层

Supervisor 根据意图识别结果，将问题分发到对应的专家 Agent：

Agent	名称	处理意图	核心能力	工具数
Query Agent	取数专家	data_query / lineage / table_meta / table_code / sql_dev / mixed	5 种模式分路：血缘溯源、取数、元数据、源代码、SQL 开发	~23
Analysis Agent	分析专家	analysis / attribution	波动归因诊断树 + ReAct 自由探查 + Python 沙箱	~20
Ops Agent	运维专家	ops	调度实例、日志、报警、基线查询	~10
Other Agent	兜底	other	寒暄、闲聊	~3

为什么不用单 Agent：单 Agent 需要一个巨大的 system prompt 和几十个工具，导致工具选择困难、prompt 膨胀、调用预算浪费。多 Agent 让每个专家只看自己相关的工具和指令，选择更精准。

4.3 ReAct 推理循环：执行层

Agent 不是"一条命令出结果"，而是**边想边做**的 ReAct 循环。以"昨天 GMV 和前天比怎么样？"为例：

```
Thought 1: 需要先找到 GMV 所在的表
Action 1:  search_metrics(query="GMV")
Result 1:  → ads_flow_sum_di, gmv_amt
Thought 2: 知道表了，查 SQL 规则确认口径
Action 2:  rule_search(table="ads_flow_sum_di")
Result 2:  → 必须加 WHERE activity_type=''
Thought 3: 生成 SQL 查昨天和前天
Action 3:  build_sql(table="ads_flow_sum_di",
                     metrics=["gmv_amt"],
                     bizdate_start="2026-06-14",
                     compare_bizdate_start="2026-06-13")
Result 3:  → SELECT ... (完整 SQL)
Thought 4: 执行 SQL
Action 4:  odps_query(sql="...")
Result 4:  → 昨天 XX亿，前天 XX亿，环比 -20%
Thought 5: 用户问了"怎么样"，需要分析环比
Action 5:  python_analyze(code="df['环比'] = ...")
```

```
Result 5:  → 环比下降 20%，主要受家电行业影响
Final Answer: 昨天 GMV 1.2 亿，环比下降 20%...
```

为什么用 ReAct 而不是固定流程：

问题类型	固定流程	ReAct
"昨天 GMV"	固定 5 步，简单问题也走完	2 步搞定，自主收敛
"GMV 为什么跌了，是不是家电"	固定 8 步，无法适配	自主决定下钻方向
"对比各行业 GMV 做帕累托"	无法预定义	取数 + 分析，5 步完成

ReAct 让 Agent **自主决定**调什么工具、传什么参数、什么时候收敛。

4.4 安全守卫：保障层

Agent 有"刹车系统"（Harness），防止失控：

规则	说明	严重度
R2 死循环检测	连续 6 次调用同一工具 → 告警	warn
R5 表验证	生成 SQL 前必须先查元数据	warn
R7 预聚合表保护	禁止手写预聚合表 SQL（必须用 builder）	warn
R8 精确去重	相同工具 + 相同参数不重复执行	warn
R9 总量上限	软上限 30 次 warn / 硬上限 50 次 fatal	warn→fatal

此外，所有 Agent 生成的 SQL 都经过 **SQL Validator Gate** 后置校验，确保语法正确、表名存在、字段合法。

05 NL2MDL2SQL：从自然语言到正确 SQL

> *这是整个系统**最关键的技术选型**。*

5.1 为什么不直接 Text-to-SQL

直接让 LLM 写 SQL 有三个致命问题：

```
直接 Text-to-SQL
  │
  ├── ❌ 幻觉字段：LLM 编造不存在的列名
  ├── ❌ 口径错误：不知道某表的 WHERE 条件该加什么
  └── ❌ 不可复用：每次从零生成，没有沉淀
```

LLM 不知道 `ads_bybt_flow_sum_di` 表必须有 `activity_type = '百亿补贴'` 这个 WHERE 条件。它会编造字段，会漏掉过滤条件，每次生成结果还不一样。

5.2 WrenAI MDL 语义层

WrenAI 引入了 **MDL（Model Definition Language）**——类似 dbt 的语义建模层：

```
用户问题
  │
  ▼
WrenAI MDL 语义层
  │
  ├── mdl.json（模型定义文件）      ← 表名、字段、关系、描述
  ├── dry_plan（NL → SQL 预演）     ← 自然语言转 SQL
  ├── recall（查询历史召回）         ← 记住之前问过什么
  └── fetch_context（上下文召回）    ← 给 LLM 补充表结构信息
  │
  ▼
生成的 SQL 经过语义层校验
  │
  ▼
✅ 字段存在    ✅ 口径正确    ✅ 可复用
```

5.3 MDL 模型结构

`mdl.json` 定义了**每个表的结构化语义**：

```
{
  "models": [
    {
      "name": "ads_flow_sum_di",
"tableReference": { "table": "ads_flow_sum_di" },
      "columns": [
        { "name": "gmv_amt", "type": "decimal", "expression": "pay_amt'
        { "name": "bizdate", "type": "date" }
      ],
      "description": "汇总日表"
    }
  ]
}
```

Agent 在生成 SQL 前会先从 MDL 获取表结构，确保只用真实存在的字段。

5.4 SQL 构建的三种模式

Agent 根据表类型自动选择 SQL 构建策略：

模式	触发条件	工作方式
预聚合表 Builder	表是 ROLLUP/CUBE 类型	自动补全维度过滤，防止漏维度
普通表 Builder	表是 dwd/dws/dim 类型	自动注入 snippet 和规则条件
WrenAI NL2SQL	传 question 参数，复杂多表 JOIN	LLM 基于语义层直接生成

5.5 取数链路中的知识分层

4.3 节展示了 ReAct 的 Thought→Action→Result 机制。在取数场景中，这些工具调用可以按知识层级归为四个阶段：

阶段	调用工具	对应知识层
知识检索	search_metrics / cap_search	Layer 1 指标定义 + 能力清单
知识应用	rule_search / snippet_search	Layer 2 片段库 + Layer 3 规则库
语义层生成	build_sql	WrenAI MDL
执行与分析	odps_query / python_analyze	—

Agent 在 ReAct 循环中自主决定调用顺序和是否需要每个阶段——简单取数可能跳过知识应用，复杂分析可能多轮调用。

5.6 Hologres 外部表加速

数据分析场景涉及多次递归下钻查询，每次走 ODPS 执行 SQL 耗时 30-120 秒。当 Agent 需要执行 5-10 次 SQL 才能完成一次诊断时，总耗时可能超过 10 分钟。引入 Hologres 外部表直读 ODPS，将单次查询加速到 3-10 秒。

为什么选 Holo 外部表而不是全量同步：

方案	优点	缺点
全量同步（T+1 导入）	查询最快	需 DBA 维护同步链路，增量管理复杂
外部表直读	零运维，数据实时	首次查慢 1-3s（建外部表），后续走 Holo 引擎

选择外部表直读：不需要 DBA 介入，onboard 新表自动挂载，第一次查询时自动创建外部表，后续 LRU 缓存命中。

场景闸门——只在数据分析场景启用：

Holo 加速仅对 `stream_analysis`（多步分析编排器）生效，普通 NL2SQL、CLI 脚本、运维查询一律走 ODPS。通过 `contextvars.ContextVar` 实现线程/协程安全的场景标记：

`HOLO_ROUTE_MODE`	`holo_scope`	结果
`off`	*	ODPS（kill switch，一键关闭）
`always`	False	ODPS（非数据分析场景）
`always`	True	Holo（数据分析场景）

执行链路：

```
LLM 生成 ODPSSQL
  │
  ▼
odps_query 工具（接口签名不变）
  │
  ├─ Step0: 场景闸门——HOLO_ROUTE_MODE=always 且 holo_scope=True?
  │         否 → 直走 Step7ODPS
  │         是 ↓
  ├─ Step1: 变量替换（${bizdate} → 具体日期值）
```

```
├─ Step2: 黑名单检测（GET_JSON_OBJECT/EXPLODE 等不兼容函数 → 直走 ODPS）
├─ Step3: 抽取 SQL 中所有表名（sqlglot AST 解析）
├─ Step4: ensure_foreign_tables——按需创建外部表
│         检查 information_schema.foreign_tables 是否已存在
│         不存在 → IMPORTFOREIGNSCHEMA <project> LIMITTO (<table>)
│         LRU 缓存 + 持久化 JSON，避免重复 DDL
├─ Step5: sqlglot 方言转换（ODPS → PostgreSQL）
│         + 表名重写（bm_dw.xxx → public.xxx）
├─ Step6: 通过 aistudio HTTP 网关执行
│         ├─ 成功 → 返回结果（标记 "via Hologres"）
│         └─ 失败 → Step7
└─ Step7: FallbackODPS（PyODPS execute_sql，原路径）
```

外部表自动管理：

不需要 DBA 预先 IMPORT 整个 project（上千表一次性 IMPORT 很贵）。而是拿到 SQL 后用 sqlglot 抽出表名，按需创建。首次查某表多 1–3 秒建表开销，后续查 LRU 命中零开销。

```
# 进程内存缓存（set）+ 持久化文件（.holo_foreign_imported.json）
# 幂等去重，重启不丢失
# 外部表与 ODPS 表同名，默认 prefix/suffix 为空
```

切库自愈：

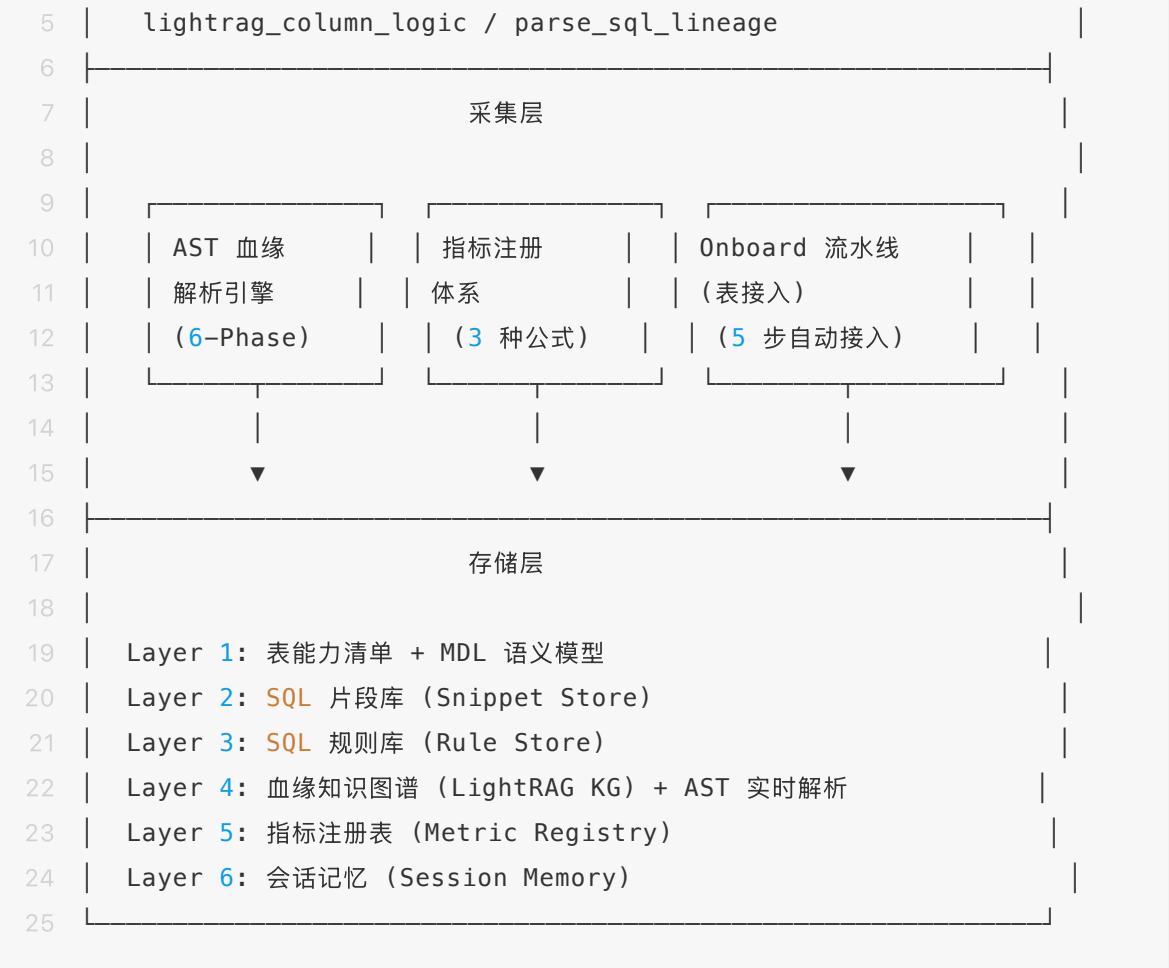
当 Hologres 实例切库（如重建实例）后，所有外部表丢失。此时查询会报 `relation "xxx" does not exist`。系统自动检测该错误，清空 IMPORT 缓存，重新 ensure 外部表后重试一次。整个过程对 Agent 透明，最多增加 3 秒。



知识库是 Agent 的"记忆"。一个能正确取数、能深度归因的 Agent，背后必须有一套结构化的知识体系支撑。本章从**存储**、**采集**、**应用**三个维度展开。

6.1 知识体系全景

```
┌──────────────────────────────────────────────────────────┐
│                        应用层                            │
│   Agent ReAct 循环主动调用知识检索工具                    │
│   search_metrics / cap_search / snippet_search / rule_search  │
```



6.2 六层存储体系

知识分为六层，每层有不同的粒度、变更频率和权威等级：

层级	名称	内容	底座	变更频率
Layer 1	表能力清单 + MDL	表结构、可用维度、指标、适用场景	TisPlus3 + mdl.json	表结构变更时
Layer 2	SQL 片段库	已审核的标准 SQL 模板，可复用	TisPlus3 odps_sql_snippets	每次对话产出
Layer 3	SQL 规则库	每张表的 WHERE 条件、聚合口径、字段选取规则	TisPlus3 odps_sql_rules	表变更时
Layer 4	血缘知识图谱	表→列→上游列的完整血缘关系	LightRAG + TisPlus3 + AST 引擎	ETL 变更时
Layer 5	指标注册表	指标定义、拆解公式、维度分类	metrics.yaml + formula_registry	新指标接入时
Layer 6	会话记忆	跨 Session 的用户偏好、常用表、上下文	TisPlus3 + DashScope Embedding	每次对话

权威等级：

层级	权威等级	说明
已审核 SQL 片段的 WHERE 条件	硬约束	不得修改，必须原样使用
规则库的规则	必须遵循	每条规则是表级编写规范
指标公式注册表	诊断依据	归因诊断树的拆解公式来源
参考级 SQL 片段	结构参考	可借鉴结构，允许调整
MDL 模型	字段基准	只能用 MDL 中定义的字段

6.3 AST 血缘解析引擎

血缘知识（Layer 4）的数据从哪来？答案是 **AST 血缘解析引擎**——一套基于 sqlglot 的 SQL 静态分析管线，从 ETL 源代码中自动提取字段级血缘。

- 为什么需要 AST 解析？

血缘关系是数据治理的核心：某张表的 GMV 字段来自哪几张上游表？经过了什么计算？WHERE 条件是什么？这些信息分散在数百行的 ETL SQL 中，人工维护不可持续。

- 六阶段解析管线

```
原始 ETL SQL
  │
  ▼
Phase 0: Schema Registry        ← 加载所有相关表的字段定义
  │                                 (本地 JSON 文件 + DataWorks API 降级)
  ▼
Phase 1: 预处理 (Preprocess)    ← 全角标准化 / DDL 剥离 / 变量替换
  │                                 多 INSERT 拆分 / 方言兼容
  ▼
Phase 2: AST 标准化 (Normalize) ← sqlglot 解析 SQL → AST
  │                                 展开 SELECT * 为显式列列表
  ▼
Phase 3: 节点提取 (Extract)     ← DFS 遍历 AST，按 scope 提取
  │                                 CTE / 子查询 / UNION 分支 / 主查询
  │                                 每个.scope: 源表、输出列、JOIN、WHERE
  ▼
Phase 4: 作用域解析 (Resolve)   ← 符号表 + alias 消歧
  │                                 CTE scope 隔离，拓扑序逐层解析
  │                                 裸列名 → schema 消歧定位
  ▼
Phase 5: 血缘映射 (Map)         ← resolved scopes → 血缘边 (Edge)
```

```
  │                                  direct: 上游列直传
  │                                  computed: 表达式中的所有列引用
  │                                  aggregated: SUM/COUNT 等聚合
  ▼
Phase 6: 输出构建 (Output)          ← 生成标准化 JSON
                                     target_table / source_tables /
                                     columns(含 depends_on) / joins /
                                     where_conditions
```

- 两种使用模式

模式	场景	工具	说明
离线 KG 构建	Onboard 新表时	`lineage_kg_builder.py`	解析 ETL SQL → 生成三元组 → 灌入 LightRAG 知识图谱
实时解析	Agent 对话中	`parse_sql_lineage`	用户贴一段 SQL 或从 DataWorks 取源代码后即时解析

- 实时解析的降级链

```
Agent 需要查字段血缘
  │
  ├── 1. lightrag_column_logic(table, column)  ← 先查 LightRAG KG
  │       ↓ KG 未命中（表未 onboard）
  ├── 2. dw_get_table_source(table)             ← 取 ETL 源代码
  │       ↓ 拿到 SQL
  └── 3. parse_sql_lineage(sql, column)         ← AST 实时解析
          ↓ 返回字段映射、来源表、WHERE 条件
```

- 解析能力覆盖

引擎能处理复杂 ODPS SQL 语法，包括：

语法结构	处理方式
CTE (WITH 子句)	独立 scope，exported_columns 接口通信
UNION ALL	每个分支独立 scope，对齐列位置
子查询	递归提取，parent-child 关联
ROLLUP / CUBE / GROUPING SETS	结构化提取，保留语义
LATERAL VIEW EXPLODE	识别产出列和源列
UNPIVOT	列转行，识别值列和名称列
TRANSFORM (Hive UDF)	流式 UDF，识别输入列和输出列
ODPS 专有函数	TO_CHAR/TO_DATE 保全，防止 sqlglot 退化

- 准确性保障

内置 **回归测试框架**，覆盖 45 张真实表、296 个节点：

校验维度	抽样量	错误率	95% 置信上界
computed_alias 滥用反查	809	0.00%	—
同名列归属冲突	2000	0.00%	—
depends_on 表完整性	230	0.00%	—
JOIN/WHERE 归属校验	1348 + 1413	0.00%	—
GROUP BY 归属	1619	0.00%	0.94%
UNION 列对齐	760	0.00%	0.50%
综合准确率	**7848**	**100.00%**	—

6.4 指标注册体系

指标注册表（Layer 5）是波动归因诊断的核心知识源——它定义了每个业务指标**如何拆解**。

- 为什么需要指标注册？

归因诊断的核心是"拆解"：GMV 跌了 20%，要拆成访客数 × 转化率 × 客单价，找出哪个因子是主因。但"GMV 等于什么"这个知识从哪来？

如果靠 LLM 自己猜，它可能拆成"流量 × 客单价"——漏掉了转化率。如果写死在代码里，每加一个指标都要改代码。

指标注册体系让指标定义成为**声明式配置**，运营同学通过 Web 向导就能接入新指标。

- 三种公式类型

类型	语义	适用场景	示例
additive (加法型)	指标 = Σ(各维度值之和)	可直接累加的绝对值指标	商品数、补贴金额、流量 PV
multiplicative (乘法型)	指标 = A × B × C	漏斗关系指标	GMV = 曝光UV × 转化率 × 客单价
ratio (除法型)	指标 = 分子 / 分母	比率/效率类指标	ROI = GMV / 补贴金额

- MetricDef 结构

以 GMV 为例，一个完整的指标定义：

```yaml
metric_id: gmv
name: GMV
formula_type: multiplicative
description: 支付GMV，流量漏斗拆解为 曝光UV × 转化率 × 客单价
total_col: pay_amt
factors:
  - name: IPVUV
    label: 曝光UV
    type: column              # 直接取列值
    column: ipv_uv
  - name: CVR
    label: 转化率
    type: ratio               # 分子/分母
    numerator: ord_cnt
    denominator: ipv_uv
  - name: AOV
    label: 客单价
    type: ratio
    numerator: pay_amt
    denominator: ord_cnt
source:
  table: ads_sup_index_di
dimensions:
  - dim_col: ind1_name
    label: 行业
    perspective: goods        # 供给/商品类
    axis: business
  - dim_col: act_type
    label: 活动类型
    perspective: place        # 渠道/流量类
```

```
    axis: traffic
aliases: [gmv_amt, 交易额, 成交额]
```

乘法型恒等式铁律：因子相乘后约分必须恰好等于 total_col。

```
GMV = IPVUV × CVR × AOV
    = ipv_uv × (ord_cnt / ipv_uv) × (pay_amt / ord_cnt)
    = pay_amt  ✅
```

- 维度分类体系

每个指标关联一组可拆解维度，维度按两个正交轴分类：

perspective (视角)	含义	示例维度
people	人群/用户类	buyer_type, user_tag
goods	供给/商品类	biz_type, ind1_name, item_type
place	场域/渠道类	act_type, channel, flow_name

axis (分析轴)	含义
funnel	漏斗层
user	用户侧
business	供给/招商
traffic	流量/渠道
supply	供给侧

- AI 辅助接入流程

指标接入通过 Web 平台的"指标接入向导"完成，AI 辅助生成 MetricDef：

```
1. 选择 MDL 表              → 系统列出表结构和字段
2. AI 生成 MetricDef        → LLM 根据 schema 自动推导公式类型和因子
3. 预览生成的公式           → 校验恒等式、维度分类
4. 确认入库                 → 写入 metrics.yaml + formula_registry
5. 自动同步                 → 刷新 metric_registry 缓存
```

AI 生成时的容错机制：

- perspective/axis 混淆自动修正（LLM 常把 axis 值填到 perspective）
- Pydantic schema 逐条校验，不合法的标红但不阻断合法项
- 注入 table_profile 信息辅助 AI 理解 ROLLUP/CUBE 结构

- 指标与归因诊断的关系

指标注册表直接驱动归因诊断树（详见第七章）：

```
metrics.yaml（指标定义）
  │
  ▼
formula_registry（公式展开）
  │  GMV → [漏斗公式: IPVUV × CVR × AOV]
  │        [加法公式: Σ(各行业 GMV)]
  │        [比率公式: GMV / 总流量]
  ▼
归因诊断引擎
  │  多路筛选 → 选最显著拆解
  │  递归下钻 → 沿主因子逐层拆
  ▼
诊断树报告
```

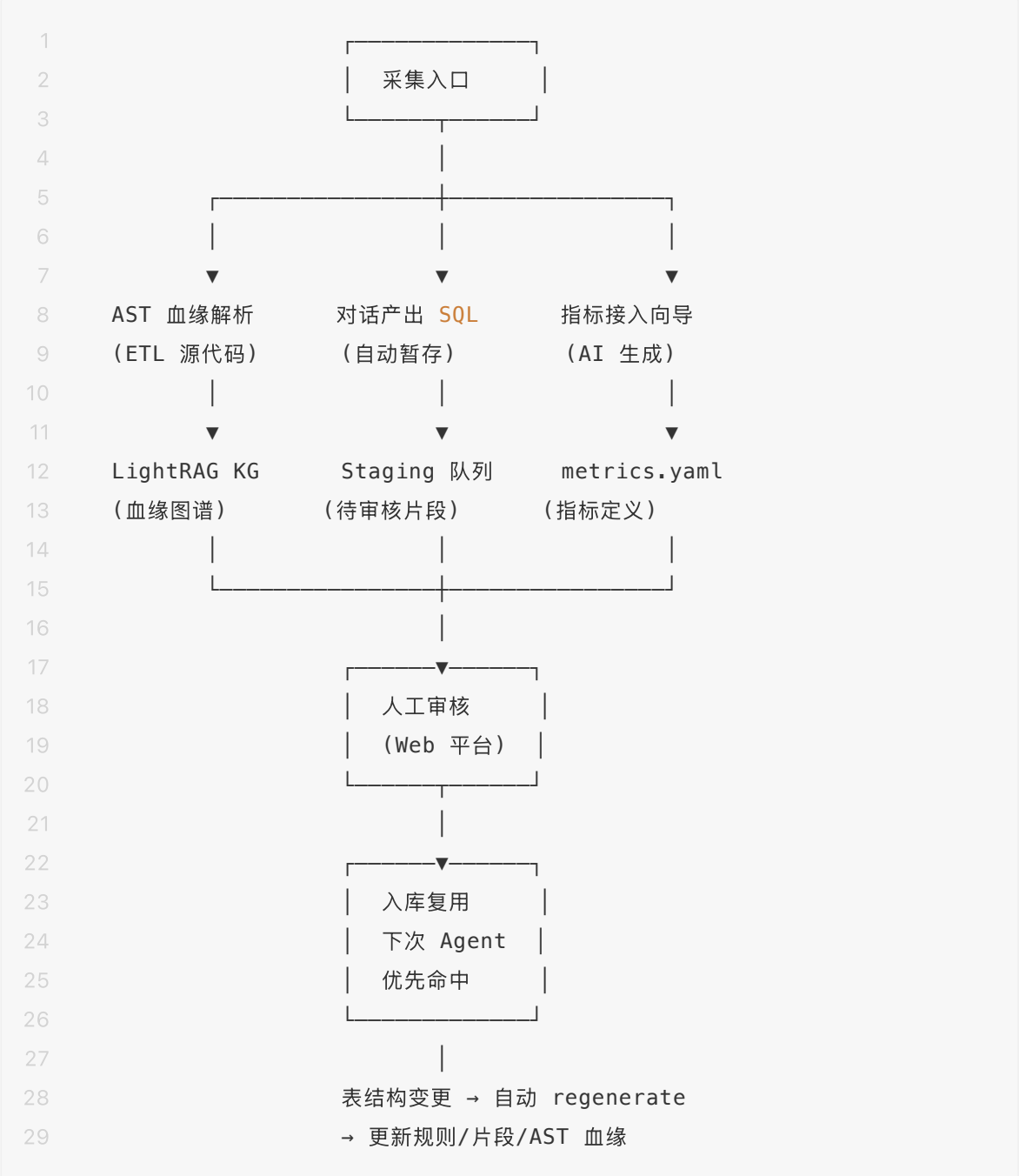
6.5 知识在 ReAct 中的使用

Agent 在 ReAct 循环中会**主动调用**知识检索工具，每层知识对应不同的工具：

```
Agent 思考："用户问 GMV，我需要先找表"
  → search_metrics("GMV")            ← Layer 5: 指标定义（找到 gmv_amt）
  → cap_search("GMV ")           ← Layer 1: 能力清单（找到表）
  → snippet_search(table="ads...")  ← Layer 2: SQL 模板
  → rule_search(table="ads...")     ← Layer 3: 编写规则
  → lightrag_table_lineage("ads...")← Layer 4: 血缘关系（KG）
  → parse_sql_lineage(sql="...")    ← Layer 4: 血缘关系（AST 实时）
```

6.6 采集与治理闭环

知识不是静态的，而是**采集** → **沉淀** → **审核** → **复用**的闭环：



Web 平台提供完整的知识管理界面：

页面	能力
表管理	表清单、onboard、regenerate、字段查看
SQL 片段评审	查看/编辑/审核 SQL 模板
规则评审	查看/编辑/审核编写规则
暂存队列	对话产生的 SQL 确认入库
指标接入向导	AI 生成 → 预览 → 入库 → 同步
对话回放	SSE 事件流回放、工具调用展开

7.1 波动归因诊断（Analysis Agent）

用户视角：

> *“昨天 GMV 为什么跌了 20%？”*

Agent 自动构建诊断树：

```
GMV 环比 -20%
  │
  ├── 漏斗拆解：GMV = 访客数 × 转化率 × 客单价
  │    ├── 访客数 +5%    ← 不是原因
  │    ├── 转化率 -22%   ← ⭐ 主因！
  │    └── 客单价 +3%    ← 不是原因
  │
  ├── 转化率 -22% 按行业拆
  │    ├── 家电 -45%     ← ⭐ Top1 贡献
  │    ├── 服装 -8%
  │    └── 食品 +2%
  │
  ├── 家电转化率 -45% 指标拆解
  │    ├── 下单量 -40%   ← ⭐ 主因
  │    └── 访问量 -8%
  │
  └── 结论：家电行业下单量骤降导致 GMV 下跌
```

技术实现五步法：

步骤	说明	知识来源
1. 公式注册表	读取指标的拆解公式，如 GMV = 曝光UV × 转化率 × 客单价	Layer 5: 指标注册表
2. 多路筛选	同时尝试多个公式，选出最显著的拆解维度	formula_registry
3. 递归下钻	沿主因子逐层拆维度，直到收敛（变化率 < 阈值）	—
4. 自适应阈值	根据数据波动特征动态调整显著性判定	—
5. ReAct 自由探查	公式不覆盖的场景，Agent 自主选工具探查	—

7.2 血缘溯源

用户视角：

> *“GMV 这个字段是怎么计算出来的？上游依赖哪些表？”*

Agent 的血缘溯源有两条路径，形成 **KG 优先、AST 降级**的降级链：

```
# 路径 1: 查 LightRAG 知识图谱（local 模式）
lightrag_column_logic(table="ads_flow_sum_di", column="gmv_amt")
→ gmv_amt = pay_amt（直接映射）
  上游: dwd_ord_ent.pay_amt
  过滤: WHERE activity_type =''
# 路径 2: KG 未命中时，AST 实时解析 ETL 源代码
dw_get_table_source(table="ads_flow_sum_di")  → 拿到 SQL
parse_sql_lineage(sql_text="INSERT OVERWRITE TABLE ...", target_column=
→ gmv_amt = pay_amt
  source_columns: [dwd_ord_ent.pay_amt]
  where_conditions: [activity_type ='']
# 查表的完整上游依赖（hybrid 模式）
lightrag_table_lineage(table="ads_flow_sum_di")
→ 上游依赖 3 张表:
1. dwd_ord_ent（交易明细）
2. dim_activity_info（活动维表）
3. dws_flow_sum_h（小时汇总）
```

KG 中的血缘数据由 AST 引擎在 onboard 时离线解析并灌入，实时解析作为降级补充。详见 6.3 节。

7.3 Python 数据分析沙箱

当查询结果需要**二次加工**时（帕累托、排名、透视、环比计算），Agent 会调用 `python_analyze`：

```
# Agent 生成的分析代码（沙箱内执行）
df['环比'] = (df['today'] - df['yesterday']) / df['yesterday']
df = df.sort_values('today', ascending=False)
df['累计占比'] = df['today'].cumsum() / df['today'].sum()
result = df[['industry', 'today', 'yesterday', '环比', '累计占比']].head(
print(result.to_markdown())
```

安全模型：

防护	说明
白名单 import	只允许 pandas/numpy/math 等安全库
禁用危险 builtins	屏蔽 `open` / `exec` / `eval` / `__import__`
超时控制	单次执行超过阈值自动终止
自动注入	`df` 自动注入为上次查询结果



8.1 新表接入流程（Onboard）

新表接入是一个自动化的五步流水线：

```
1. 表结构同步     → ODPS schema → mdl.json + TisPlus3
2. 表类型探查     → A类(预聚合)/B类(普通) + 主键 + 维度
3. 规则抽取       → 下游血缘代码 → LLM 提取 WHERE 条件
4. SQL 模板生成   → B类表自动生成标准模板
5. 入库           → 规则/片段/能力清单写入 TisPlus3
```

支持 Web 界面操作和 CLI 脚本两种方式。

8.2 项目结构

```
bybt-data-analysis-assistant/
├── agent.py                # CLI 入口 + Agent 构建
├── agent_analysis.py       # 模板分析入口
├── config.py               # 全局配置（ODPS/LLM/TisPlus3/...）
├── boot.sh                 # 生产部署脚本
│
├── agents/                 # 多 Agent 架构
│   ├── supervisor.py       # 路由调度器
│   ├── intent_agent.py     # 意图识别 Agent
│   ├── base_expert.py      # 专家 Agent 基类
│   ├── query_agent.py      # 取数+血缘+元数据+源代码+SQL开发
│   ├── analysis_agent.py   # 波动归因+数据分析
│   ├── ops_agent.py        # 调度运维
│   ├── other_agent.py      # 兜底寒暄
│   ├── harness.py          # 运行时安全守卫
│   └── sql_validator_gate.py  # SQL 后置校验
│
├── tools/                  # 工具集（Agent 的"手"）
│   ├── odps/               # ODPS 查询 + SQL 构建
│   ├── capabilities/       # 表能力清单检索
│   ├── snippets/           # SQL 片段检索
│   ├── rules/              # SQL 规则检索
│   ├── lineage/            # 血缘知识图谱
│   ├── analysis/           # Python 分析沙箱
│   ├── attribution/        # 归因诊断引擎
│   ├── dataworks/          # DataWorks API
│   ├── memory/             # 会话记忆
│   └── intent/             # 意图识别
│
├── web/                    # Web 平台
│   ├── api/                # FastAPI 后端
│   └── frontend/           # React 前端
│
└── wren_project/           # WrenAI MDL 语义模型
    ├── target/mdl.json     # 模型定义文件
    └── .wren/memory/       # 向量索引
```

评测体系

Agent 的「聪明」不能靠感觉判断，必须有可量化、可复现的评测体系。本章回答三个核心问题：**评什么**（6D 维度）、**怎么构造样例**（Golden Dataset）、**怎么打分**（意图感知权重 + 节点归因）。

9.1 6D 评分维度

评测从 6 个维度对 Agent 的每次回答打分，每个维度 0–100 分：

维度	名称	评什么	适用意图
D1	因子命中率	归因诊断树中找到的主因子是否与期望一致	ANALYSIS
D2	诊断树结构	下钻深度、分支数、方向是否达标	ANALYSIS
D3	工具合规性	该调的工具调没调、不该调的调没调、调用顺序对不对	ALL
D4	结论质量	LLM-as-Judge 按意图特定 Rubric 打分（量化数据/因果链/可执行性）	ALL
D5	SQL 正确性	SQL 语法、表名、字段名、WHERE 条件、白名单校验	DATA_QUERY / LINEAGE / MIXED
D6	忠实度	回答中的事实声明是否都能被工具结果或知识库支撑（幻觉检测）	ALL

9.2 意图感知权重

不同意图关注点不同——归因分析不关心 SQL 正确性（D5=0），取数不关心因子命中（D1=0）。系统为每个意图配置了差异化权重：

意图	D1	D2	D3	D4	D5	D6	侧重点
analysis	0.25	0.20	0.20	0.15	—	0.20	因子 + 树 + 工具
data_query	—	—	0.25	0.20	**0.35**	0.20	SQL + 工具
lineage	—	—	0.20	**0.30**	0.20	**0.30**	结论 + 忠实度
mixed	—	—	0.20	0.20	**0.30**	**0.30**	SQL + 结论 + 忠实
table_meta	—	—	0.25	0.25	—	**0.50**	忠实度（防幻觉字段）

> *权重为 0 的维度显示为 `--`，不参与打分。权重可通过 Web 界面自定义调整（`scripts/eval/metrics_config.json`）。*

9.3 Golden Dataset — 样例构造

评测样例存放在 `scripts/eval/datasets/default.json`，当前覆盖 **6 类意图、15 个用例**：

类型	用例数	典型问题	期望检查点
归因分析	2	"GMV波动分析"	主因子命中、公式调用、诊断树深度、方向正确
取数查询	3	"查昨天大盘GMV"	SQL 包含正确表名、字段白名单校验、预聚合过滤
血缘溯源	3	"动销商品数怎么计算的"	调用 lightrag + dw_get_table_source、结论含计算表达式
混合意图	2	"GMV口径是什么并查昨天的值"	两段式回答（口径 + 结果）、SQL + 血缘工具都调用
表元数据	2	"有哪些字段"	调用 cap_get/cap_search、结论含字段名
边界用例	2	错误字段名 / 不存在的表	能识别错误并纠正、给出提示

单个样例结构示例：

```
{
"id":"gmv_daily_attribution",
"question":"20260610 GMV波动分析",
"intent":"analysis",
"tags":["attribution","gmv","daily"],
"expected":{
"main_factor":["3C数码","坑产","平台出资","GMV_PER_ITEM"],
"main_factor_fuzzy":["3C","坑产","平台"],
"formula_used":["gmv.industry","gmv.supply","gmv_per_item.is_bt"],
"direction":"up",
"min_tree_depth":2,
"must_call_tools":["attribution_diagnose","attribution_explain"],
"must_not_call_tools":[],
"conclusion_must_contain":["贡献","主因","3C"]
}
}
```

样例设计原则：

- **expected 多层校验**：不只看最终回答，还看中间过程（工具调用序列、SQL 内容、诊断树结构）
- **模糊匹配兜底**：`main_factor_fuzzy` 允许部分命中得 60 分，避免因措辞差异误判
- **意图覆盖**：每类意图至少 2 个用例，边界用例覆盖错误输入场景
- **可扩展**：通过 Web 界面「评测中心」可在线增删用例，无需改代码

9.4 评测执行流程

```
用户提问
  │
  ▼
harness.run_case(question)
  │  调用 stream_supervised() 获取 SSE 事件流
  │  收集: tool_call / tool_result / token / routing / validation 事件
  │  解析: attribution_diagnose 返回的诊断树 JSON
  │  检测: HOLO fallback 标记
  ▼
EvalTrace（结构化 trace）
  │  events: 全部 SSE 事件
```

```
   │  tool_calls: 工具调用序列
   │  tool_results: 工具返回结果
   │  diagnosis_tree: 诊断树 JSON
   │  final_answer: 最终回答
   │  nodes: 7 个关键节点的状态
   ▼
6D 打分 (scorers)
   │  D1 factor_scorer     → 严格匹配(100) / 模糊匹配(60) / 方向匹配(30) / 不
   │  D2 tree_scorer       → 深度 + 分支数 + 方向
   │  D3 tool_scorer       → must_call(40) + must_not_call(25) + orderin
   │  D4 conclusion_scorer → LLM-as-Judge 按意图 Rubric 打分
   │  D5 sql_scorer        → 语法 + 表名 + 字段名 + WHERE + 白名单
   │  D6 faithfulness_scorer → 规则检查 + LLM 幻觉检测
   ▼
加权总分 = Σ(Di × Wi)  →  意图感知权重
   ▼
报告输出
   ├── 终端彩色报告 (6D 分数 + 归因诊断)
   ├── JSON 报告 → reports/eval_<timestamp>_<git_hash>.json
   ├── Bad Case 归档 → badcases/ (总分<60 或任一维度=0)
   └── 版本对比 → --compare old.json new.json
```

9.5 节点级归因诊断

评测不只给分数，还自动定位**失败根因**。Harness 从事件流中提取 7 个关键节点的状态：

节点	数据来源	fail 条件
intent	`intent` 事件	意图分类为空
routing	`routing` 事件	未收到路由事件
tool_call	`tool_result` 事件	工具返回 ❌ 开头
sql_gen	`build_sql` 结果	返回错误
gate	`validation_*` 事件	重试耗尽未通过
answer	最终回答	回答为空或执行异常

当总分低于预期时，归因诊断会指出是哪个节点出了问题，例如：

```
根因: gate – validation_exhausted (重试 3 次未通过)
根因: tool_call – 工具返回错误: odps_query
```

```
3  根因: quality – 各节点均成功，但回答质量未达预期
```



本文作者伯略，来自淘天集团–天猫技术团队。百亿补贴与聚划算是淘天面向价格敏感用户与品牌商家的核心营销阵地，也是我们直面市场竞争最前沿的“主战场”；我们不断“刷新”技术能力，以半托管、竞价与补贴系统的建设，支撑百亿补贴成为淘天近三年增长最快的业务。如今我们正用 AI 重构从招商审核到补贴发放的运营全流程，推动“人工配置”向“AI 自治”演进，并把研发链路重塑为需求分析、代码生成到测试上线的端到端 AI 原生范式。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

964篇原创内容

公众号

阅读 9127

留言

写留言

还没有留言，来写首评